//

Recunoaşteţi următoarele expresii şi precizaţi tipul de date(atom sau listă):

((atom a (3) elem))

3

(3)

)(

(((a b c)

Atom

(atom elem))))

- >>>>Lista formata dintr-o lista,format din 4 elemente,dintre care primul al doilea si al 4-ulea sunt atomi simbolici iar al 3 este o lista formata dintr-un atom numeric.
->>>>Atom numeric.
->>>>Lista formata dintr-un atom numeric.
->>>>Expresie incorecta.
->>>>Expresie care nu poate fi inca evaluata,deoarece numarul de paranteze din drepata trebuie sa fie egal sau mai mare cu cel din stanga.
->>>>Atom simbolic.
->>>>Lista formata din doi atomi simbolici,paranteleze in plus din dreapta sunt ignorate.

//

Precizaţi numărul de elemente ale următoarelor liste, şi apoi tipul elementelor:

((1 3) 3.4 (a b c) ((atom)))

((a (b c)) d)

((((a))) ((b)) (c) d)

(a b cd)

->>>>Lista formata din 4 elemente,dintre care primul al treilea si al patrulea sunt liste iar al doilea este un atom numeric(numar real).
->>>>Lista forma din 2 elemente ,primul fiind o lista,iar al doilea un atom simbolic.
->>>>Lista formata din 4 elemente,primul al doilea si al treilea fiind liste,iar al patrulea un atom simbolic.
->>>>Lista formata din 3 atomi simbolici.
//

Scrieți exemple de liste :

Cu 4 elemente atomi de tip numeric

Cu 3 elemente de tip listă

Cu 2 elemente atomi simbolici si 2 elemente de tip listă, fiecare cu câte 2 elemente atomi numerici

(1 2 3 4)
((1) (1 2 3) (1 2 3 4))
(a b (1 2) (3 4))
//

Transpuneţi în LISP următoarele expresii şi apoi evaluaţi-le:

(25 + 30) * 15/2

6 * 3.1416

(* (+ 25 30) (/ 15 2))
825/2
(* 6 3.1416)
18.8496
//

Determinaţi câtul intreg pentru media aritmetică a numerelor:

5, 6.7, -23.2, 75 si 100.3

(floor (/ (+ 5 6.7 -23.2 75 100.3) 5))
32
//

Determinaţi restul împărţirii numerelor:

365 şi 12, 13 şi 467

(mod 365 12)
5
(mod 13 467)
13
//

Determinaţi rezultatul întreg prin rotunjire în jos, respectiv în sus, apoi incrementaţi , respectiv decrementaţi rezultatul pentru următoarea expresie,:

(5 * 2.25 – 7.13) / (45 – 25 / 5)

(floor (/ (- (* 5 2.25) 7.13) (- 45 (/ 25 5))))
0

```
 (ceiling (/ ( - (* 5 2.25) 7.13) (- 45 (/ 25 5))))
1
 (1+ (floor (/ ( - (* 5 2.25) 7.13) (- 45 (/ 25 5)))))
1
 (1-(floor (/ ( - (* 5 2.25) 7.13) (- 45 (/ 25 5)))))
-1
(1+(ceiling (/ ( - (* 5 2.25) 7.13) (- 45 (/ 25 5)))))
2
(1-(ceiling (/ ( - (* 5 2.25) 7.13) (- 45 (/ 25 5)))))
0
//
```

Evaluaţi următoarele forme:

(* (MAX 3 4 5) (MIN 3 4 5))

(EXPT (MAX 3 1 4) (MAX 2 7 1))

(REM (+ 5 7 13) (- 14 1))

```
 (*  (MAX 3 4 5)  (MIN 3 4 5))
15
 (EXPT   (MAX 3 1 4)  (MAX 2 7 1))
16384
(REM  (+  5  7  13) (- 14  1))
12
//
```

Afişaţi rezultatul de la problema 3 sub forma unui mesaj:

Ex: restul impărțirii numerelor ... şi ... este:...

```
 (format t " Restul impartirii numerelor ~A si ~A este ~A" 365 12 (/ 365 12))
 Restul impartirii numerelor 365 si 12 este 365/12
NIL
 (format t " Restul impartirii numerelor ~A si ~A este ~A" 13 467 (/ 13 467))
 Restul impartirii numerelor 13 si 467 este 13/467
NIL
```

Afişați un mesaj înainte de citirea unei variabile, apoi citiți variabila.

Ex: “a =” citire valoare

```
(format t "a=")(read)
a=12
//
```

1. Scoateţi simbolul D din următoarele expresii folosind CAR si CDR.

(A B C D)
(A (B C) (D E))
(A (B C (D E)))
((A) (B) (C) (D) (E))
(A (B) ((C)) (((D))) ((((E)))))
((((A) B) C) D)

(car(cdr(cdr(cdr'(A B C D)))))
(car(car (cdr (cdr '(A (B C) (D E))))))
(car(car(cdr(cdr(car(cdr '(A (B C (D E))))))))
(car(car(cdr(cdr(cdr '((A) (B) (C) (D) (E)))))))
(car(car(car(car(cdr(car(cdr(cdr '(A (B) ((C) (((D))) ((((E))))))))))))))
(car(cdr '((((A) B) C) D)))

2. Evaluaţi următoarea expresie Lisp, apoi returnaţi lungimea listei rezultat:

(subst ' azi ' maine (reverse ' (frumoasa zi o este maine)))

(subst' azi' maine(reverse ' (frumoasa zi o este maine)))
(length(subst' azi' maine(reverse ' (frumoasa zi o este maine))))

3. Evaluaţi în ordine şi argumentaţi rezultatul, următoarele expresii :

(setq lista '(a b c (d e))
(setq M 'max)
(cons M lista)
(cons lista M)
(append (list M) (last (cdr lista)))
(list ' M (cadr lista))
(append ' lista (list M (last lista)))
(list (car lista) (cadr lista) (caddr lista) M)

(setq lista '(a b c (d e)))
(A B C (D E))
(cons lista M)

((A B C (D E)) . MAX)
(append (list M) (last(cdr lista)))
(MAX (D E))
(list 'M (cadr lista))
(M B)
(append 'lista (list M(last lista)))
; Evaluation aborted on #<TYPE-ERROR expected-type: LIST datum: LISTA>.
(list (car lista) (cadr lista) (caddr lista) M)
(A B C MAX)

4. Se considera lista:

((A) (B) (C) (D) (E))

Folosind toate functiile de constructie a listelor si cele de acces la elementele unei liste, contruiti urmatoarea lista:

((((A) (B) (C)) . B) (E D C) D)

Se evalueaza cu nota maxima acea rezolvare care include toate functiile predefinite prezentate.

(setq l '((A) (B) (C) (D) (E)))
(setq l1 (list (car l) (cadr l) (caddr l)))
(setq l2 (cons l1 (cadr l)))
(setq l3 (list (caar(last l)) (car(cadddr l)) (caaddr l)))
(append (list l2 l3) (cadddr l))

5. Se considera lista:

l1 : ((((A) (B)) C) (D) (EF G))

Corectati eroarea aparuta in urma evaluarii expresiei:

(append (cadr l1) (car (nth '1 l1)) (last l1))

Argumentati apoi rezultatul evaluarii expresiei:

(cons (nth '3 l1) (nthcdr '2 l1))

(append (CADR L1) (CAR (nth '1 l1)) (last l1))
-> (car (nth ‘1 l1)) returneaza un atom si nu o lista,iar la append doar ultimul argument poate fi un atom,restul doar liste
(cons (nth '3 l1) (nthcdr '2 l1))
(NIL (EF G))

6. Evaluând expresiile urmatoare, obţineţi rezultatele:

```
> (setq  E  '(x y ((z) u))  F  (last  E)  G  (caar F))
(Z)
>(setq  X  '12  Y  '14  Z  '22  E (-  (+  x  y)  z))
4
>(setq  M  (MIN 1  -4  23  56  1)  P  (* M  10)  X  (EXPT  P  2))
1600
```

Evaluaţi pentru fiecare expresie în parte simbolurile şi argumentaţi rezultatul.

```
(setq E'(x y ((z) u)) F (last  E)  G  (caar F))
 E
(X Y ((Z) U))
 F
(((Z) U))
 G
(Z)
 (setq X  '12  Y  '14  Z  '22  E (-(+  x  y)  z))
 X
12
Y
14
 Z
22
 E
4
//
```

7. Se considera lista:

(A B C D)

Folosind functia SUBST impreuna cu alte functii, obtineti din lista initiala lista:

(D B C A)

Folosind functia REVERSE impreuna cu alte functii, obtineti din lista initiala, lista:

(D A B C)

```
(subst (car(last l)) 'X (cons 'X (subst 'A 'D (cdr l))))
(reverse (append (cdr (reverse l)) (last l)))
```

//

1. Exprimați cu ajutorul lui if modulul unui număr şi operatorul not:

(abs x) (not y)

```
(defun modul(x)(if (< x 0) (* x -1) x))
(defun negat (x)(if (equal x nil) t nil))
```

//

2. Exprimați cu ajutorul lui cond : and, or între 3 argumente:

(or x y z) (and u v w)

```
(defun sau (x y z) (cond ((not(equal x nil)) x) ((not(equal y nil)) y) (t z)))
(defun si (u v w) (cond ((equal u nil) nil) ((equal v nil) nil) (t w)))
```

//

3. Determinați cu ajutorul lui cond elementul din mijloc dintre trei elemente numerice.

```
(defun mij (a b c) (cond ((or (not(numberp a)) (not(numberp b)) (not(numberp c))) nil) (t b)))
```

//

4. Definiți funcțiile *my-third, my-last,* care să returneze al trei-lea element al unei liste, respectiv ultimul element al unei liste.

```
(defun my-third (l) (cond ((< (length l) 3) nil) (t (car(cdr(cdr l))))))
(defun my-last (l) (cond ((not(listp l)) nil) (t (car (last l)))))
```

//

5. Definiți o funcție *my-append* care primeşte ca argumente doi parametri de tip listă şi returnează o listă de lungime dublă obținută prin concatenarea celor două liste inițiale. Apoi definiți predicatul *palindrom* care determină dacă lista astfel obținută este sau nu palindrom.

```
(defun my-append (l1 l2) (cond ((or (not(listp l1)) (not(listp l2))) nil) (t (append l1 l2))))
(defun palindrom (l) (if (equal l (reverse l)) t nil))
```

//

6. Definiți o funcție care primeşte ca parametri coeficienții unei ecuații de gradul 2 şi care dacă delta este pozitiv returnează o listă cu soluțiile ecuației, dacă delta este egal cu 0, returnează soluția unică a ecuației de gradul 2, iar dacă delta este negativ, returnează mesajul "solutii complexe".

```
(defun delta(a b c) (eval (- (* b b) (* 4 a c))))
(defun ec (a b c) (cond ((or (not(numberp a)) (not(numberp b)) (not(numberp c))) nil)
                        ((> (delta a b c) 0) (cons (/ (+ (* b -1) (sqrt (delta a b c))) (* 2 a)) (/ (- (* b
-1) (sqrt (delta a b c))) (* 2 a))))
                        ((= (delta a b c) 0) (/ (* b -1) (* 2 a)))
                        (t "solutii complexe")))
```

//

7. Definiți o funcție care primeşte un singur argument x şi returnează valoarea unei funcții definte astfel:

$$f(x) = \begin{cases} x^2; x \le -2 \\ x+2; -2 < x \le 1 \\ \sqrt{x^2+1}; x > 1 \end{cases}$$

```
(defun fx (x) (cond ((<= x -2) (* x x))
                    ((and (<= x 1) (> x -2)) (+ x 2))
                    ((> x 1) (sqrt (+ (* x x) 1)))))
```

//

8. Definiți o funcție care primeşte ca argumente un atom şi o listă, şi returnează o listă în care se adaugă la coada listei inițiale atomul, numai în cazul în care atomul nu se regăseşte în listă. In caz contrar, se returnează fragmentul de listă începând cu elementul găsit.

```
(defun fc (a l)(cond ((not(atom a)) nil)
                     ((not(listp l)) nil)
                     ((member a l) (member a l))
                     (t (append l a))))
```

//

9. Definiți o funcție care afişează un meniu interactiv, citeşte câte un caracter şi execută o operație conform caracterului citit, iar în cazul în care caracterul nu este nici unul dintre caracterele citite, returnează un mesaj de eroare.

1- Adunare
2 – Scadere
3 – Inmulțire
4 – Restul împărțirii

Se considera trei variabile inițializate cu valori citite de la tastatură , primele Două reprezintă două numere intregi, iar ultima, numarul corespunzător operației ce va fi executată. În corpul lui LET se execută operația corespunzătoare caracterului citit pentru cea de-a treia variabilă.

```
(defun meniu ()
    (format t "Introduceti un valorile")
    (let ((a (read)) (b (read)) (ca (read)))
    (cond
    ((equal ca '1) (+ a b))
    ((equal ca '2) (- a b))
    ((equal ca '3) (* a b))
    ((equal ca '4) (/ a b))
    (t "eroare"))))
```

//

1.(1p)Să se implementeze recursiv o funcţie COUNTATOMS care primeşte ca argument o listă ce conţine şi liste imbricate şi returnează numărul total de elemente ale listei .

```
(defun countatoms (l nr)
    (cond ( (null l) nr)
```

```
        ((atom l) (+ nr 1))
        (t (countatoms (cdr l) (+ nr (countatoms (car l) 0))))))
```

//

2.(2p)Să se implementeze funcțiile recursive *Rot-left* şi *Rot-right* care rotesc la stânga, respectiv la dreapta n elemente ale unei liste.
Rotire la stânga înseamnă mutarea primului element al listei la coada listei.
Rotire la dreapta înseamnă mutarea ultimului element al listei în fața listei.

```
(defun rot-left (l n nr lis)
        (cond ((= nr 0) (rot-left (cdr l) n (+ nr 1) (car l)))
        ((= nr n) (cons lis l))
         (t (cons (car l) (rot-left (cdr l) n (+ nr 1) lis)))))
 (defun rot-right(l n nr lis)
        (cond ((= nr (- n 1)) (cons (car l) (append lis (cdr l))))
         (t (rot-right (cdr l) n (+ 1 nr) (append lis (cons (car l) nil))))))
```

//

3. (1p)Scrieţi o funcţie *fără _dubluri (lista)* care returnează lista fără dubluri. Elementele să fie în aceeaşi ordine în lista returnată ca şi în lista iniţială, păstrându se ultima apariţie a elementelor duplicat.
Exemplu: > (f a r a _d u b l u r i ' (1 2 2 3 4 5 6 4 4)) → (1 2 3 5 6 4)

```
(defun fara_dubluri (l)
        (cond ((null l) ())
         (t (cond ((member (car l) (cdr l)) (fara_dubluri(cdr l)))
                        (t (cons (car l) (fara_dubluri(cdr l))))))))
```

//

4.(1p)Implementați recursiv predicatul *presentp* care determină dacă un atom apare oriunde în interiorul unei liste, chiar şi o listă ce conține liste imbricate.

```
(defun presentp (a l)
        (cond ((null l) nil)
         ((atom l) (= a l))
         (t (or (presentp a (car l)) (presentp a (cdr l))))))
```

//

5.(1p)Implementați recursiv o funcție care primeşte ca argument un număr întreg şi returnează lista cu cifrele numărului.

```
(defun lista (nr)
        (cond ((= nr 0) nil)
         (t (cons (mod nr 10) (lista (floor(/ nr 10)))))))
```

```
(defun create (nr)
        (cond ((zerop nr) '(0))
          (t (reverse(lista nr))))
        )
```

//

6.(1p) Implementați iterativ funcția pozpar, care primeşte ca argument o listă şi returnează o listă cu elmentele de pe pozițiile impare ale listei inițiale.

```
(defun pozpar(lst)
        (do ((l lst (cdr l)) (nr 0 (+ 1 nr)) (x '() ))
        ((null l) x)
        (cond ((= (mod nr 2) 0) (setq x (append x (cons (car l) nil))))
                (t x))))
```

//

7.(2p)Implementați iterativ o funcție , care determină cel mai mare divizor comun a două numere întregi. Evidenţiaţi parametri actuali şi valoarea rezultată a funcţiei apelate, în ordinea apelurilor, respectiv terminărilor funcţiei pentru un exemplu concret.

```
(defun cmmdc(a b)
        (do ((c a )(d b ))
        ((= c d) c)
        (cond((> c d) (setq c (- c d)))
          (t (setq d (- d c))))))
```

//

1. (1p)Sa se implementeze o functie care returneaza lista formata din toate cifrele impare ale unui numar intreg.

```
(defun impare (nr l)
        (cond((= nr 0)  l)
         ((= (mod (mod nr 10) 2) 1) (impare (floor (/ nr 10)) (cons (mod nr 10) l)))
         (t (impare (floor (/ nr 10)) l))))
```

//

2. (1p)Sa se scrie o functie recursiva care primeste ca argument 2 liste si returneaza lista de lungime dubla avand ca elemente, elementele celor 2 liste intercalate.(!!listele initiale se considera de lungimi egale).

```
(defun interc (l1 l2)
        (cond((null l1) nil)
```

//

3. (2p) Sa se scrie o functie care primeste ca argument o lista generalizata si returneaza lista a carei elemente sunt lungimile fiecarei subliste a listei initiale, elementele de tip atom nu se iau in considerare, daca lista nu contine elemente de tip lista se returneaza lista vida.

```
(defun lungl (l)
        (cond ((null l) nil)
          ((listp (car l)) (cons (length (car l)) (lungl (cdr l))))
          (t (lungl (cdr l)))))
```

//

4. (2p)Sa se implementeze o funcție care primeste ca argument o lista liniara cu elemente numerice si un numar intreg N și returnează lista in care apar de două ori elementele din N în N.

Exemplu:

Pentru lista:

(1 2 3 4 5) si N=2 -> (1 2 2 3 4 4 5)

```
(defun dubl (l n nr)
        (cond ((null l) nil)
          ((= (mod nr n) 0) (cons (car l) (cons (car l) (dubl (cdr l) n (+ nr 1)))))
          (t (cons (car l) (dubl (cdr l) n (+ nr 1))))))
```

//

5. (3p) Implementaţi iterativ reuniunea, intersecţia, diferenţa si egalitatea a 2 mulţimi.

```
(defun reun (m1 m2)
        (do ((l1 m1 ) (l2 m2  (cdr l2)))
        ((null l2) l1)
        (cond ((not(member (car l2) l1)) (setq l1 (cons (car l2) l1))))))
(defun inter (m1 m2)
        (do ((l1 m1 ) (l2 m2 (cdr l2)) (l nil ))
        ((null l2) l)
        (cond ((member (car l2) l1) (setq l (cons (car l2) l))))))
(defun diff (m1 m2)
        (do ((l1 m1 (cdr l1)) (l2 m2 ) (l nil ))
        ((null l1) l)
        (cond ((not(member (car l1) l2)) (setq l (cons (car l1) l))))))
(defun egal (m1 m2)
        (do ((l1 m1 (cdr l1)) (l2 m2 (cdr l2)) (v t ))
        ((and (null l1) (null l2)) v)
```

```
        (cond((not(equal (car l1) (car l2))) (setq v nil)))))
//
```

1.(1p) Implementaţi o funcţie care adună toate elementele unei liste, aflate pe poziţii pare . Poziţiile se consideră începând cu 0.

```
(defun lispozpar(l p)
        (cond((null l) nil)
         ((= (mod p 2) 0) (cons (car l) (lispozpar (cdr l) (+ p 1))))
         (t (lispozpar (cdr l) (+ p 1)))))
(defun sumpozpar (l)
        (apply '+ (lispozpar l '0)))
//
```

2.(3p) Implementaţi recursiv reuniunea,intersecţia şi diferenţa dintre 2 mulţimi folosind remove-if-not.

```
(defun reun (l1 l2)
        (cons l1 (diff l2 l1)))
(defun inter (l1 l2)
        (remove-if-not (lambda (x) (member x l2)) l1))
(defun diff (l1 l2)
        (remove-if-not (lambda (x) (not(member x l2))) l1))
//
```

3.(1p) Definiţi o funcţie care numără apariţiile unui număr dat într-o listă generalizată, folosind mapcar şi lambda.

```
(defun numar-egal (l nr)
        (cond ((atom l) (cond((equal l nr) 1) (t 0)))
          (t (apply '+ (mapcar (lambda(x) (numar-egal x nr)) l)))))
//
```

4.(4p) Definiţi o funcţie care afişează o listă generalizată sub forma precizată pentru exemplul următor:

(a b (c d) (e f g) h) →

```
( a
  b
  ( c
    d )
      ( e
        f
        g )
  h )
```

```
(setq pars `|{|)
(setq pard `|}|)
(setq space `| |)
(defun spatii (x) (cond ((null (zerop x)) (princ space) (spatii (- x 1)))))
(defun afis_restul1 (lst nivel par) (cond
                                ((null (cdr lst)) (aux1 lst nivel par) (princ space) (princ pard) (terpri))
                                ((atom (car lst)) (aux lst nivel par))
                                (t (aux3 lst nivel 1))))
(defun afis_primul (l nivel par) (spatii nivel) (princ pars) (princ space) (princ (car l)) (terpri)
(afis_restul1 (cdr l) (+ nivel 2) par))
(defun aux (lst nivel par) (if (equal par 0) (spatii 2)  (spatii nivel)) (princ (car lst)) (terpri)
(afis_restul1 (cdr lst) nivel par))
(defun aux1 (lst nivel par)  (if (equal par 0) (spatii 2) (spatii nivel)) (princ (car lst)))
(defun aux3 (lst nivel par) (afis_primul (car lst) (+ 2 nivel) 1) (afis_restul1 (cdr lst) (+ nivel 2) 0))
(afis_primul `(1 2 3 (4 5) (6 7) 8) 0 0)
//
```

1. Scrieţi o funcţie FETCH care primeşte o cheie şi o listă de asociaţii, iar dacă găseşte cheia va returna doar valoarea ei. Dacă nu o găseşte va returna un mesaj corespunzător

```
> (fetch 'temperature '((temperature 100)
                        (pressure (120 60))
                        (pulse 72)))
100
```

```
(defun fetch (v s)
```

```
        (cond((equal (assoc v s) nil) (print '|Nu exista|))
         (t (cadr (assoc v s)))))
```
//

2. Scrieţi o funcţie ce returnează o listă cu toate cheile dintr-o listă de asociere.

```
(defun lista (l)
        (cond ((null l) nil)
          (t (cons (caar l) (lista (cdr l))))))
```
//

3. Presupunând că avem simboluri ce au proprietatea **tata**. Definiţi procedura bunic ce primeşte un simbol şi returnează bunicul din partea tatălui dacă este cunoscut, alftel nil.

```
 (setf (get 'mihai 'tata) 'sergiu)
(setf (get 'sergiu 'tata) 'paul)
(setf (get 'mihai 'mama) 'ioana)
(Setf (get 'sergiu 'mama) 'iulia)
(setf (get 'paul 'tata) 'rares)
(defun bunic (s)
        (get (get s 'tata) 'tata))
```
//

4. Definiţi funcţia recursivă ADAM care primeşte un simbol şi returnează cel mai îndepărtat strămoş pe linie paternă.

```
(defun adam (s)
        (cond ((equal (get s 'tata) nil) s)
          (t (adam (get s 'tata)))))
```
//

5. Definiţi funcţia recursivă STRAMOSI care returnează o listă formată din persoana împreuna cu toţi strămoşii cunoscuţi, mergând pe două proprietatăti: tata şi mama.

```
 (defun stra (s l)
        (cond ((null(or (get s 'tata) (get s 'mama))) l)
        ((null (get s 'tata)) (stra (get s 'mama) (append (list (get s 'mama)) l)))
        ((null (get s 'mama)) (stra (get s 'tata) (append (list (get s 'tata)) l)))
        (t (append (stra (get s 'tata) nil) (append (stra (get s 'mama) nil) (append (list (get s 'tata)
(get s 'mama)) l))))))
(defun stramosi (s)
        (append (stra s nil) (list s)))
```
//

6. Definiţi un macro define care defineşte o funcţie având sintaxa:
(define <nume functie> (<param 1> ... <param n>) <corp>)

```
(defmacro define (nf l c)
        `(defun ,nf ,l ,c))
```

//

7. Definiţi un macro dotimes cu sintaxa:
(dotimes (<var> <count> <rezultat> <corp>)

Prima dată se evaluează <count> care trebuie să fie un întreg. Apoi <var> este succesiv legata la întregi de la 0 la valoarea lui <count> minus 1. Corpul este evaluat de fiecare dată iar <rezultat> este returnat la sfârşit.

```
(defmacro dotimes (v c r co)
        `(do ((,v 0 (+ ,v 1)))
         ((= ,v ,c) ,r)
        ,co))
```

//

1. Scrieţi o funcţie PRINT-ARRAY care afişează o matrice de două dimensiuni primită ca argument. Fiecare rând al matricii va fi afişat pe o linie nouă. Folosiţi ARRAY-DIMENSION pentru a afla domeniul indicilor.

```
(setq m (make-array '(3 3)))
(setf (aref m 0 0) 1)
(setf (aref m 0 1) 2)
(setf (aref m 0 2) 3)
(setf (aref m 1 0) 4)
(setf (aref m 1 1) 5)
(setf (aref m 1 2) 6)
(setf (aref m 2 0) 7)
(setf (aref m 2 1) 8)
(setf (aref m 2 2) 9)
(defun print-array (m)
(do((linie 0 (+ 1 linie)))
        ((= linie (array-dimension m 0) ))
        (do((coloana 0 (+ 1 coloana)))
        ((= coloana (array-dimension m 1)))
```

```
        (cond(t (princ (aref m linie coloana))(princ '| |)))

        )(terpri)
))
(print-array m)
//
```

2. Scrieţi câte o funcţie care transformă o matrice reprezentată ca listă de liste, în ambele moduri prezentate, într-o matrice standard LISP. Determinaţi apoi numărul elementelor pare ale matricii.

```
(setq m '((1 2 3) (4 5 6) (7 8 9)))
(setq m1 '((0 0 1) (0 1 2) (1 0 3) (1 1 4)))
(defun setvac (l ma i j)
        (cond ((not(null l)) (setf (aref ma i j) (car l)) (setvac (cdr l) ma i (+ j 1)))))
(defun setval (l ma i)
        (cond((not(null l)) (setvac (car l) ma i 0) (setval (cdr l) ma (+ i 1)))))
(setq ma (make-array `(,(length m) ,(length (car m)))))
(setval m ma 0)
ma
(defun cm2 (l ma)
        (cond ((not (null l))(setf (aref ma (caar l) (cadar l)) (car(last (car l))))(cm2 (cdr l) ma))))
(setq ma1 (make-array `(,(+ 1 (caar (reverse m1))) ,(+ 1 (cadar (reverse m1))))))
(cm2 m1 ma1)
ma1
(defun numara-pare (m)
  (
  do
  (
    (linie 0 (+ linie 1))
    (nr 0)
  )
  ((equal linie (array-dimension m 0)) nr)
  (
    do (
    (coloana 0 (+ coloana 1))
    )
    ((equal coloana (array-dimension m 1)) nr)
    (cond ((equal (mod (aref m linie coloana) 2) 0) (setq nr (+ nr 1)))
  )
```

```
   )))
(numara-pare ma)
(numara-pare ma1)
//
```

3. Definiţi o structură de tip nod cu elementele: rădăcina stga dr;

   Implementaţi apoi o funcţie care adaugă un nou nod, după regula: în stânga pentru valorile <= decât rădăcina, sau în dreapta pentru valori mai mari decât rădăcina. Creaţi un arbore .

```
(defstruct nod rad st dr)
(defun creaza (x) (make-nod :rad x :st nil :dr nil))
(defun adauga (n r) (
   cond ((null r) (setf r n))
   ((< (nod-rad n) (nod-rad r))
      (if (null (nod-st r)) (setf (nod-st r) n)(adauga n (nod-st r))))
   (t (if (null (nod-dr r)) (setf (nod-dr r) n)(adauga n (nod-dr r))))
))
(setq n1 (creaza 3))
(setq n2 (creaza 5))
(setq n3 (creaza 1))
(setq n4 (creaza 7))
(setq n5 (creaza 2 ))
(adauga n2 n1)
(adauga n3 n1)
(adauga n4 n1)
(adauga n5 n1)
n1
//
```

Problema 1

Implementaţi în ML funcţia lui fibonacii.

```
# let rec fibbo =function n ->
#     if n=0 then 0 else
#        if n=1 then 1 else
#           fibbo (n-1) + fibbo (n-2);;
```

```
val fibbo : int -> int = <fun>
# fibbo 3;;
- : int = 2
# fibbo 4;;
- : int = 3
# fibbo 5;;
- : int = 5
# fibbo 6;;
- : int = 8
```

//

Problema 2

Scrieţi funcţia `geninterval`, care primeşte doi parametri `s` şi `e` şi retunrează o listă formată din numerele întregi între `s` şi `e`.

```
# geninterval 3 8;;
- : int list = [3; 4; 5; 6; 7; 8]
```

```
let rec geninterval s e =
   if s = e  then s::[] else
      s:: geninterval (s+1) e;;
geninterval 3 8;;
```

//

Problema 3

Definiţi funcţiile `my_not`, `my_and` şi `my_or` fără a utiliza operatorii `not`, `and` sau `or`.

```
# my_and (3=3) (4=4);;
- : bool = true
```

```
let my_not e =
   if e=true then false else true;;
let my_and e1 e2=
   if e1= false then false else
      if e2=false then false else true;;
let my_or e1 e2=
   if e1= true then true else
      if e2=true then true else false;;
```

//

Problema 4

Scrieţi funcţia `digits` care transformă un număr întreg într-o listă de formată din cifrele sale.

```
# digits 123;;
- : int list = [1; 2; 3]
```

```
let rec digits nr=
   if nr<9 then nr::[] else digits (nr / 10) @ ((nr mod 10) :: []);;
```

//

Problema 5

Scrieţi funcţia `maxim` care calculează maximul dintr-o listă. Puteţi folosi funcţia `max`, care returnează maximul dintre doi parametri.

```
# maxim [2;3;1;7;5];;
- : int = 7
```

```
let rec maxim l =
   if (List.tl l = []) then (List.hd l) else
      max (List.hd l) (maxim (List.tl l));;
```

//

Problema 6

Definiţi funcţia `CMDC`, cel mai mare divizor comun

```
let rec cmdc a b =
   if a = b then a else
      if a> b then cmdc (a-b) b else
         cmdc a (b-a);;
```

//

Problema 7

Scrieţi funcţia `reverse` care inversează elementele unei liste. Nu se vor folosi `List.hd`, `List.tl` sau funcţiile de la Problema 2.

```
let rec reverse l =match l with
   [] ->[]
```

```
  | head :: tail ->(reverse tail) @ (head :: []);;
```

//

Problema 8

Scrieti funcţiile `rotate_left` şi `rotate_right` care rotesc spre stânga respectiv spre dreapta o listă. La `rotate_left` nu se vor folosi `List.hd,List.tl`

```
let rec without_last l= match l with
  []->[]
  |head::[]->[]
  |head::tail -> head :: (without_last tail);;
let rec rotate_right l =(List.hd (reverse l)) :: (without_last l);;
```

//

Problema 9

Scrieţi o funcţie `apply f i [l1::l2::..ln]` care primeşte o funcţie `f`, o valoare iniţială `i` şi o listă, şi retunează: `f(l1, f(l2, ... f(ln1, f(ln,i))...))`. Nu se vor folosi `List.hd,List.tl` sau funcţiile de la Problema 2.

```
# let u x y = x + y;;
# let f x y = 2 * x + y;;
# apply u 0 [1;2;3;4];;
- : int = 10
# apply f 1 [1;2;3;4];;
- : int = 21
```

```
let rec apply f i l =match l with
```

```
   [] -> 0
   |head :: [] -> f head i
   |head ::tail -> f head (apply f i tail)
   ;;
//
```

Problema 1.

Scrieţi funcţiile care inserează un element într-o arbore, caută un element în arbore şi returnează elementele arborelui în inordine şi preordine.

```
type 'tipelement nod=Empty
   |Nod of ('tipelement nod*'tipelement*'tipelement nod);;
let rec inserare n a=match a with
   Empty -> Nod(Empty,n,Empty)
   |Nod (Empty,v,Empty) ->if n<v then Nod(Nod(Empty,n,Empty),v,Empty) else
      Nod(Empty,v,Nod(Empty,n,Empty))
   |Nod (Empty,v,d) -> if n<v then Nod(Nod(Empty,n,Empty),v,d) else Nod(Empty,v,inserare n d)
   |Nod (s,v,Empty) -> if n<v then Nod(inserare n s,v,Empty) else Nod(s,v,Nod(Empty,n,Empty))
   |Nod (s,v,d) -> if n < v then Nod(inserare n s,v,d) else Nod(s,v,inserare n d)
   ;;
let rec cauta el a=match a with
   Empty -> "Nu exista"
   |Nod (Empty,v,Empty) -> if v = el then "Exista" else "Nu exista"
   |Nod (s,v,Empty) -> if v=el then "Exista" else if el<v then cauta el s else "Nu exista"
   |Nod (Empty,v,d) -> if v=el then "Exista" else if el<v then "Nu exista" else cauta el d
   |Nod (s,v,d) -> if v=el then "Exista" else if el<v then cauta el s else cauta el d
   ;;
let rec inordine a=match a with
   Nod(Empty,v,Empty) -> "" ^ v
   |Nod(s,v,Empty) -> (inordine s) ^ ("" ^ v)
   |Nod(Empty,v,d) -> ("" ^ v) ^ (inordine d)
   |Nod(s,v,d) -> (inordine s) ^ ("" ^ v) ^ (inordine d)
   ;;

let rec preordine a=match a with
    Nod(Empty,v,Empty) -> "" ^ v
```

```
  |Nod(s,v,Empty) -> ("" ^ v) ^(preordine s)
  |Nod(Empty,v,d) -> ("" ^ v) ^ (preordine d)
  |Nod(s,v,d) -> ("" ^ v) ^ (preordine s) ^ (preordine d)
  ;;
//
```

Problema 2.

Fiind dată o expresie prin strucura:

```
type expresie = Valoare of float
   | Adunare of (expresie*expresie)
   | Scadere of (expresie* expresie)
   | Inmultire of (expresie*expresie)
   | Impartire of (expresie* expresie)
   | Fc1 of ((float->float)*expresie)
   | Fc2 of ((float->float->float)*expresie*expresie);;
```

să se scrie expresia corespunzătoare lui `6/2-2^3+cos(2*asin 1)` precum şi o funcţie cu semnătura (expresie -> float) care evaluează orice expresie specificată prin structura de mai sus.

```
type expresie = Valoare of float
  | Adunare of (expresie*expresie)
  | Scadere of (expresie* expresie)
  | Inmultire of (expresie*expresie)
  | Impartire of (expresie* expresie)
  | Fc1 of ((float->float)*expresie)
  | Fc2 of ((float->float->float)*expresie*expresie);;

 let rec fct e =match e with
    Valoare v -> v
    |Adunare(v1,v2) -> fct v1 +. fct v2
    |Scadere(v1,v2) -> fct v1 -. fct v2
    |Inmultire(v1,v2) -> (fct v1) *. (fct v2)
    |Impartire (v1,v2)-> (fct v1) /. (fct v2)
    |Fc1 (f,v) -> f (fct v)
    |Fc2 (f,v1,v2) -> f (fct v1) (fct v2)
    ;;
let ex =Adunare(
   Scadere(Impartire(Valoare 6.0,Valoare 2.0),Fc2(rlp,Valoare 2.,Valoare 3.)),
      Fc1 (cos,Inmultire(Valoare 2.0,Fc1(asin,Valoare 1.0))))
      ;;
```

//

Sa se scrie o functie Lisp care daca o lista liniara formata din numere intregi are aspect de "vale"(o multime se spune ca are aspect de "vale"daca elementele descresc pana la un moment dat, apoi cresc) returneaza o lista formata din 3 elemente: primul element, minimul si ultimul element,iar in caz contrar Nil.

De ex: (vale '(10 8 6 9 11 13)) -> (10 6 13)
(vale '(11 2 3 6 7 9 4 1 15)) -> Nil

Să se evidenţieze paramteri actuali şi valoarea rezultată a funcţiei (funcţiilor) apelate, în ordinea apelurilor respectiv terminărilor funcţiei (funcţiilor) pentru lista din exemplu.

Punctaj: din oficiu -1p, cod -9 p, (comentarii + exemplificare in caz de nefunctionalitate -2p)

```
(defun secv_desc (l)
        (cond ((null (cdr l)) nil)
        ((< (car l) (cadr l)) (cons (car l) (cdr l)))
          (t (secv_desc (cdr l)))))
(defun secv_cresc (l)
        (cond ((null (cdr l)) t)
          ((> (car l) (cadr l)) nil)
          (t (secv_cresc (cdr l)))))
 (defun vale (l)
        (cond ((null (secv_cresc(secv_desc l))) nil)
          (t (cons (car l) (cons (car (secv_desc l)) (cons (car (last l)) nil))))))
```

//

Fiind datÄƒ o secvenÅ£Äƒ de numere Ã®ntregi Ã®n forma unei liste, sÄƒ se scrie o funcÅ£ie LISP care extrage toate secvenÅ£ele nedescrescÄƒtoare. FuncÅ£ia va returna o listÄƒ formatÄƒ din secvenÅ£ele (listele) extrase prin spargerea secvenÅ£ei iniÅ£iale.

Exemplu

(extrage '(3 8 7 8 9 9 10 3 4 6 3 1))

((3 8) (7 8 9 9 10) (3 4 6) (3) (1))

```
(defun secv_cresc (l)
(cond ((null (cdr l)) (list (car l)))
      ((<= (car l) (cadr l)) (cons (car l) (secv_cresc (cdr l))))
            (t (list (car l)))))
(defun secv(l)
(cond ((null (cdr l)) nil)
            ((> (car l) (cadr l)) (cdr l))
            (t (secv (cdr l)))))
(defun executa (l)
(cond ((null l) nil)
            (t (append(list (secv_cresc l)) (executa (secv l))))))

(print (executa '(3 8 7 8 9 9 10 3 4 6 3 1)))

//
```

ScrieÅ£i o funcÅ£ie LISP care primind ca parametru o listÄƒ returneazÄƒ o listÄƒ izomorfÄƒ, pe care dacÄƒ Ã®l liniarizÄƒm va conÅ£ine numerele Ã®ntregi de la 1 la N, unde N este lungimea listei liniarizate. Prin listÄƒ izomorfÄƒ Ã®nÅ£elegem o listÄƒ care are acelaÅŸi structurÄƒ (subliste), dar atomi din listÄƒ poate sÄƒ fie diferite, iar prin liniarizare Ã®nÅ£elegem conversia listei generalizate Ã®ntr-o listÄƒ simplÄƒ (fÄƒrÄƒ subliste cu aceeaÅŸi atomi).

Exemplu

(transforma â€˜**(s e f (f r g (3 o) i) y (i))**)

(1 2 3 (4 5 6 (7 8) 9) 10 (11))

```
(defun count-a (l)
(cond ((null l) 0)
     ((atom l) 1)
          (t (+ (count-a (car l)) (count-a (cdr l))))))
(defun trans (l i)
(cond ((null l) nil)
 ((atom l) i)
 (t (cons (trans (car l) i) (trans (cdr l) (+ i (count-a(car l))))))))

 (defun transf (l)
 (trans l 1))
//
```

Se dÄƒ o listÄƒ reprezentÃ¢nd o expresie aritmeticÄƒ (cu operatori binari +,-,*,/) Ã®n formÄƒ prefix care conÅ£ine numere Ã®ntregi ÅŸi variabile. Valorile variabilelor sunt specificate Ã®ntr-o a doua listÄƒ de forma ((variabilÄƒ1 valoare1) (variabilÄƒ2 valoare2)â€¦ (variabilÄƒn valoaren)). SÄƒ se scrie o funcÅ£ie LISP care returneazÄƒ rezultatul evaluÄƒri expresie, luÃ¢nd Ã®n considerare valorile variabilelor. ÃŽn caz cÄƒ expresia conÅ£ine variabile nedefinite se va returna nil.

Exmplu

evalexp (â€˜(* 3 (- x 1) (/ y (+ z 1))) â€˜((x 6) (y 8) (z 3)))

30

```
(defun caut (c l)
(cond ((null l) nil)
((equal c (caar l)) (cadar l))
(t (caut c (cdr l)))))

(defun verifica (c)
(cond ((equal c '+) t)
((equal c '-) t)
((equal c '*) t)
((equal c '/) t)
(t nil)))

(defun inloc (l1 l2)
(cond ((null l1) nil)
   ((atom l1) (cond ((verifica l1) l1) ((symbolp l1) (caut l1 l2)) (t l1)))
   (t (cons (inloc (car l1) l2) (inloc (cdr l1) l2)))))
 (setq m (inloc '(* 3 (+ a (- b 1))) '((a 2) (b 3))))
```

```
(defun verifica_in (l) (
            cond
            ((null l) nil)
            ((atom (car l)) (if (null (car l)) t (verifica_in (cdr l))))
            (t (or (verifica_in (car l)) (verifica_in (cdr l))))
            )
)

(defun evalueaza (l1 l2)
(if (null (verifica_in(inloc l1 l2))) (eval (inloc l1 l2)) nil))

(print(evalueaza '(* 3 (+ a (- b 1))) '((a 2) (b 3))))
//
```

Subiect 2:

-se da o lista de intregi

-scrieti o functie LISP sau ML care returneaza nr Li din L, pt care (abs(L_{i-1} - L_i)+ (abs(L_{i+1}-L_i))) este maxima, adica suma diferentelor absolute intre numar si vecinii sai de la dreapta si stanga este maxima.

-in caz de maxim multiplu, se va returna oricare dintre numerele cu valoarea maxima a diferentei.

-primul si ultimul numar al listei nu se considera

ex:

```
(difmax '(2 4 8 20 3 9 3 5 2 3))
  20
```

-pt ca abs(8-20)+abs(2-20)=12+17=29 e cea mai mare diferenta inre nr si vecinii sai

```
(defun my_left (e l)
(cond ((null (cdr l)) nil)
((= e (cadr l)) (car l))
(t (my_left e (cdr l)))))

(defun my_right (e l)
(cond ((null (cdr l)) nil)
```

```
((= e (cadr l)) (caddr l))
(t (my_right e (cdr l)))))

(defun cauta (l maxim li p)
(cond ((null (cddr l)) maxim)
((< maxim (+ (abs (- (my_left (car l) li) (car l)))
(- (car l) (my_right (car l) li)))) (cauta (cdr l) (+ (abs (- (my_left (car l) li) (car l)))
(- (car l) (my_right (car l) li))) (cdr li) (car l)))
(t (cauta (cdr l) maxim (cdr li) p))))

(defun afis (l)
(cauta (cdr l) 0 l 0))

(print (afis '(2 4 8 20 3 9 3 5 2 3)))
```

//

Subiect 1

Se dă o listă cu numere întregi şi un număr N. Să se scrie o funcţie sort_mod în LISP sau ML care returnează o listă de N liste. Sublista i din lista rezultată va conţine toate numerele la care restul împărţiri cu N este i, în ordinea în care apar în lista iniţială.

```
(defun re (nr l i)
(cond ((null l) nil)
((= (mod (car l) nr) i) (cons (car l) (re nr (cdr l) i)))
(t (re nr (cdr l) i))))



(defun rez (nr l i aux)
(cond ((= i nr ) aux)
(t (rez nr l (+ i 1) (append aux (cond ((null (re nr l i)) (list (list nil))) (t (list (re nr l i))))))))

(defun sort-mod (l nr)
(rez nr l 0 nil))
```

(print (sort-mod '(3 48 5 3 4 5 3 4 8 5 3 3 94 43 3) 5))

//

Subiect 2

Se dă o listă de L de litere, şi o listă C de liste de cuvinte (cuvintele sunt specificate ca liste de litere).
Literele fiecărei cuvânt *Cu* din lista *C* vor fi extrase pe rând din lista L. Literele cuvântului *Cu*, trebuie să fie primele litere compatibile citind de la stânga la dreapta lista L (literele alese în L trebuie să fie în acelaşi ordine ca în Cu; dacă în L sunt 2 litere identice se va alege obligatoriu prima). Să se scrie o funcţie extrage în LISP sau ML care primeşte lista L şi C, şi returnează literele din L rămase după extragerea tuturor cuvintelor din C.
Să se ruleze funcţia / funcţiile scrise cu exemplul L=(a a b b b c d e f g h i) şi C='((a b e i) (b c h)).

Exemple

```
(cuv '(a a b c ... d) '(a c))
(A B D)
(cuv '(a a b c ... d) '((a b c)(c)))
(A D)
(cuv '(a a b c ... d) '((a c)(b c)))
(A)
(cuv '(a a b c ... d) '((a b c)(c)))
NIL // c apare ca singură dată în lista 2
(cuv '(a a b c ... d) '((b a)))
NIL // pt. ca avem b nu se mai găseşte un a
(cuv '(a a c ... d e f g h i) '((a b e i) (b c h)))
(A B D F G)
```

(defun scoate (l c)

(cond ((null c) l)

((null l) nil)

((equal (car l) (car c)) (scoate (cdr l) (cdr c)))

(t (cons (car l) (scoate (cdr l) c)))))

(defun ramas (l c)

(cond ((null c) l)

(t (ramas (scoate l (car c)) (cdr c)))))

(print(ramas '(a b d c e r a c o p q s t) '((a c r o p) (b d s t))))

//

Subiect 3

Se dă o listă L de perechi formate dintr-un atom şi un contor. Scrieţi o funcţie LISP care generează o listă M. M va fi construit în modul următor: se adaugă fiecare atom din lista L în ordinea din L, după care se va decrementa contorul fiecărei atom din L şi cele care au contor mai mare ca 0 vor fi adăugate din nou în M. Procesul de adăugare-decrementare se va repeta până când toţi contorii devin egale cu 0.

Să se ruleze funcţia / funcţiile scrise cu exemplul L=((a 1) (b 4) (d 3) (e 2)).

Exemple

```
(generate '((a 2) (b 3) ) nil)
(A B A B B)
(generate '((a 2) (c 1) (b 5) ) nil)
(A C B A B B B B)
(generate '((a 1) (b 4) (d 3) (e 2)) )
(A B D E B D E B D B)
```

```
(defun verifica (l)
(cond ((null l) t)
( (equal (cadar l) 0) (verifica (cdr l)))
(t nil)))

(print(verifica '((a 0) (b 0))))


(defun decrem (v l)
(cond ((null l) nil)
((equal (caar l) v) (append (list(list (caar l) (- (cadar l) 1))) (decrem v (cdr l))))
(t (append (list(list (caar l) (cadar l))) (decrem v (cdr l))))))

(print(decrem 'a '((a 2) (b 3))))

(defun afisare (l li)
(cond ((equal (verifica li) t) nil)
((null l) (afisare li li))
((equal (cadar l) 0) (afisare (cdr l) li))
(t (cons (caar l) (afisare (cdr l) (decrem (caar l) li))))))


(print (afisare '((a 2) (b 3) (c 1)) '((a 2) (b 3) (c 1))))
```

//

Subiect 4

Se dă o listă L de perechi formate dintr-un atom şi un întreg. Să se scrie o funcţie LISP sau ML care generează o listă care cuprinde perechile formate din atomi din L şi suma tuturor întregilor care au asociate atomul curent (fiecare atom din L cu suma diferită de 0 va apărea o dată şi numai o dată în rezultat, iar cele cu suma 0 nu vor apărea).
Să se ruleze funcţia / funcţiile scrise cu exemplul L=((c 1) (b 2)(a 1)(c -1)(b 3)).

Exemple

```
(addtotal '((a 3) (b 2)(a 1)(b 6)))
((B 8) (A 4))
(addtotal '((c 1) (b 2)(a 1)(c -1)(b 3)))
((B 5) (A 1))
```

```
(defun cauta (c l)
(cond ((null l) nil)
((equal c (caar l)) (cauta c (cdr l)))
(t (cons (car l) (cauta c (cdr l))))))

(print(cauta 'a '((a 1) (b 2) (a 4) (c 3) (a 1))))

(defun suma (c l)
(cond ((null l) 0)
((equal c (caar l)) (+ (cadar l) (suma c (cdr l))))
(t (suma c (cdr l)))))

(print(suma 'a '((a 1) (b 2) (a 4) (c 3) (a 1))))

(defun rez (l)
(cond ((null l) nil)
((equal (suma (caar l) l) 0) (rez (cauta (caar l) l)))
(t (append (list(list (caar l) (suma (caar l) l))) (rez (cauta (caar l) l))))))

(print(rez  '((a 1) (b 2) (a 4) (c 3) (a -5) (b 3))))
```

//